

# MOUNT KENYA UNIVERSITY

## DEPARTMENT OF INFORMATION TECHNOLOGY

UNIT CODE: BIT4138

UNIT NAME: ADVANCED CRYPTOGRAPHY

LOGBOOK

STUDENT NAME: Samwel Abuga

Admission Number: BSCC5120241/56213

COURSE: computer science

LECTURER: Mr. Nyoro

SEMESTRER/ACADEMIC YEAR: Sem 2 year 3,

PROJECT TITTLE: SECURE FILE ENCRYPTION SYSTEM

TECHNOLOGY USED: Python + Cryptography Library

GITHUB LINK:

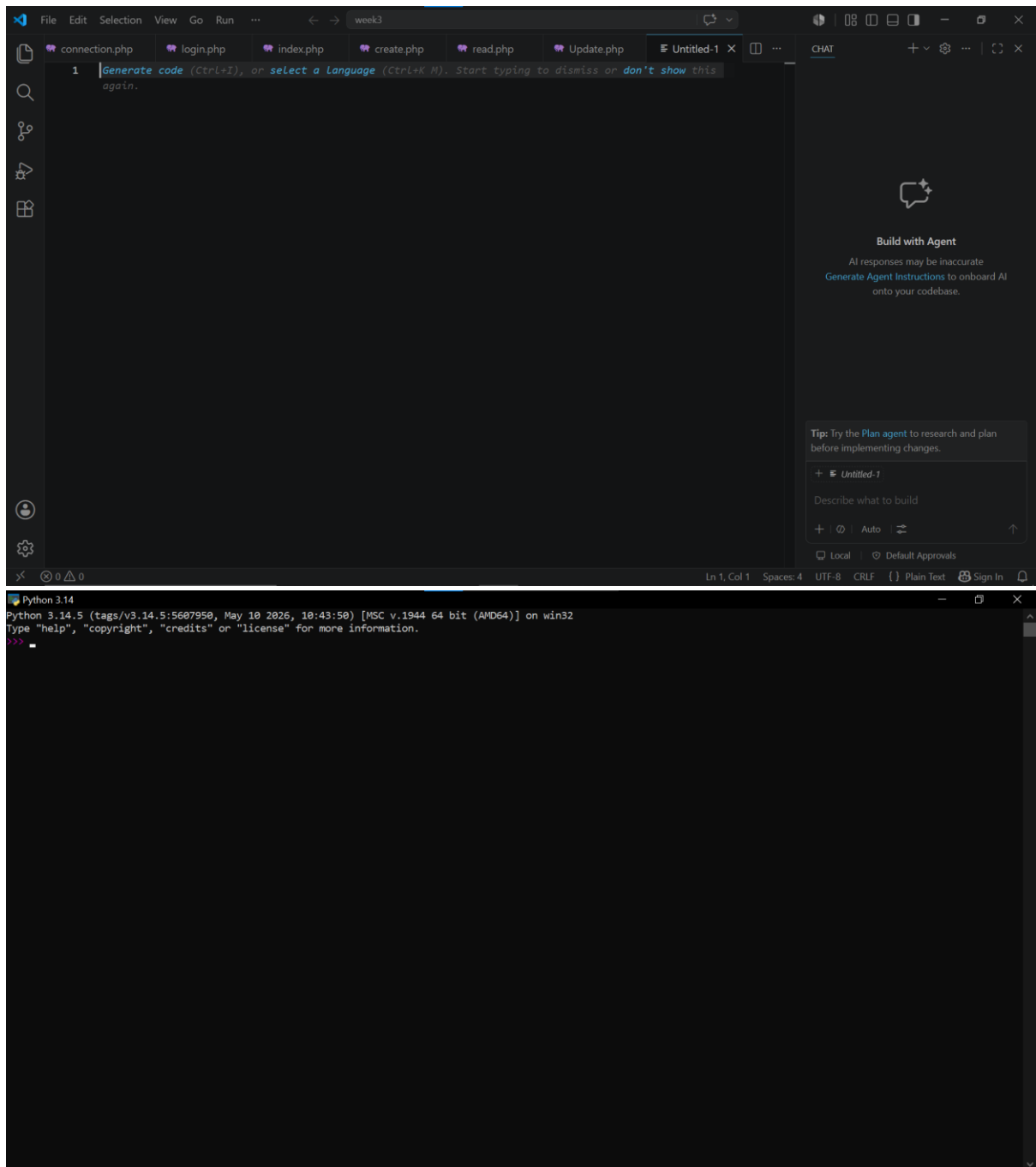
|                                                                 |   |
|-----------------------------------------------------------------|---|
| WEEK 1: .....                                                   | 1 |
| Fig 1: Installation of Python and VS Code .....                 | 1 |
| Fig 2: Installation of Cryptography Libraries.....              | 1 |
| Fig 3: OpenSSL Installation Test .....                          | 1 |
| Fig 4: Local Encryption Script Execution .....                  | 1 |
| Fig 5: GitHub Repository Initialization .....                   | 1 |
| WEEK 1 SUMMARY REFLECTION.....                                  | 1 |
| WEEK 2: IMPLEMENTATION OF CLASSICAL ENCRYPTION ALGORITHMS ..... | 1 |
| Fig 1: Caesar Cipher Implementation .....                       | 1 |
| Fig 2: Vigenère Cipher Encryption Process .....                 | 1 |
| Fig 3: Encryption and Decryption Output .....                   | 1 |
| Fig 4: User Input Validation Interface .....                    | 1 |
| Fig 5: Cipher Testing Results .....                             | 1 |
| WEEK 2 SUMMARY REFLECTION.....                                  | 1 |
| WEEK 3: IMPLEMENTATION OF STREAM CIPHER SYSTEMS .....           | 1 |
| AND RANDOM SEQUENCE ANALYSIS.....                               | 1 |
| Fig 1: LFSR Generator Implementation.....                       | 1 |
| Fig 2: Pseudorandom Sequence Output .....                       | 1 |
| Fig 3: Statistical Randomness Testing .....                     | 1 |
| Fig 4: RC4 Stream Cipher Simulation .....                       | 1 |
| Fig 5: Encryption Performance Results .....                     | 1 |
| WEEK 3 SUMMARY REFLECTION.....                                  | 1 |
| WEEK 4: ADVANCED ENCRYPTION STANDARD (AES) .....                | 1 |
| AND BLOCK CIPHER OPERATIONS.....                                | 1 |
| Fig 1: AES Encryption Script .....                              | 1 |
| Fig 2: Key Generation Process.....                              | 1 |
| Fig 3: File Encryption Demonstration .....                      | 1 |
| Fig 4: Decryption Results.....                                  | 1 |
| Fig 5: AES Performance Testing .....                            | 1 |
| WEEK 5: IMPLEMENTATION OF RSA ENCRYPTION .....                  | 1 |
| AND KEY MANAGEMENT .....                                        | 1 |
| Fig 1: RSA Key Pair Generation .....                            | 1 |

|                                             |   |
|---------------------------------------------|---|
| Fig 2: Public Key Encryption Process.....   | 1 |
| Fig 3: Private Key Decryption Results ..... | 1 |
| Fig 4: Secure Message Transmission.....     | 1 |
| Fig 5: RSA Testing and Validation.....      | 1 |
| WEEK 5 SUMMARY REFLECTION .....             | 1 |

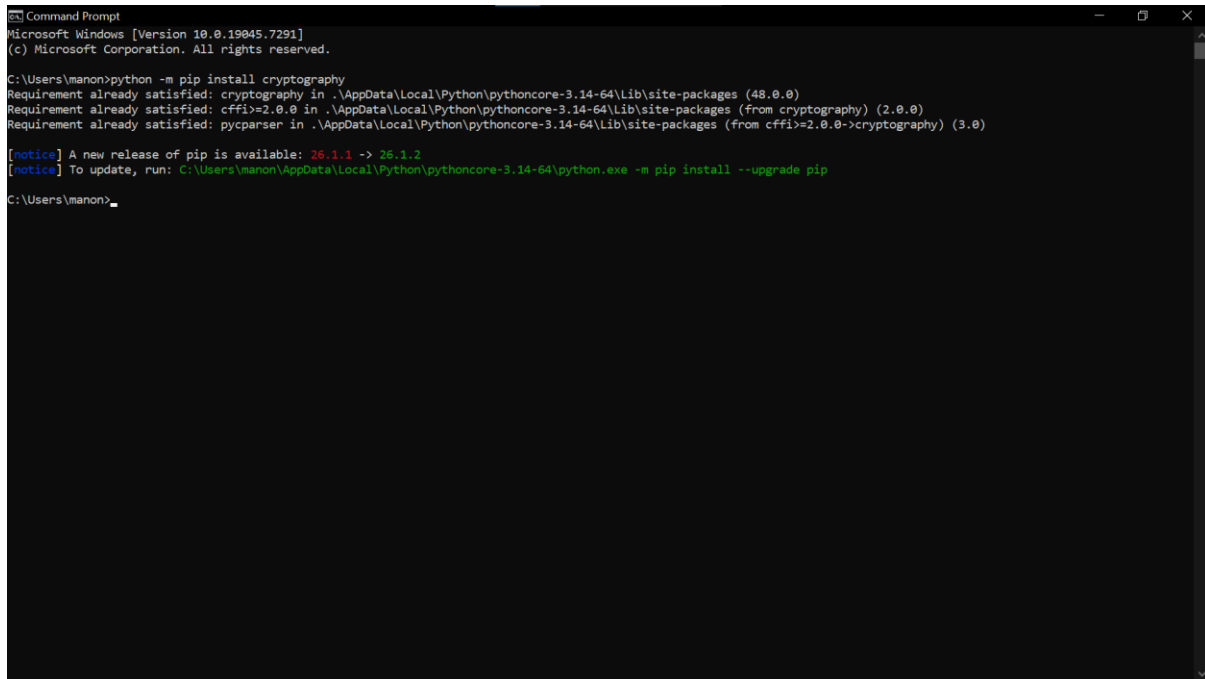
## WEEK 1:

### INSTALLATION AND CONFIGURATION OF CRYPTOGRAPHY DEVELOPMENT ENVIRONMENT

# Fig 1: Installation of Python and VS Code



## Fig 2: Installation of Cryptography Libraries



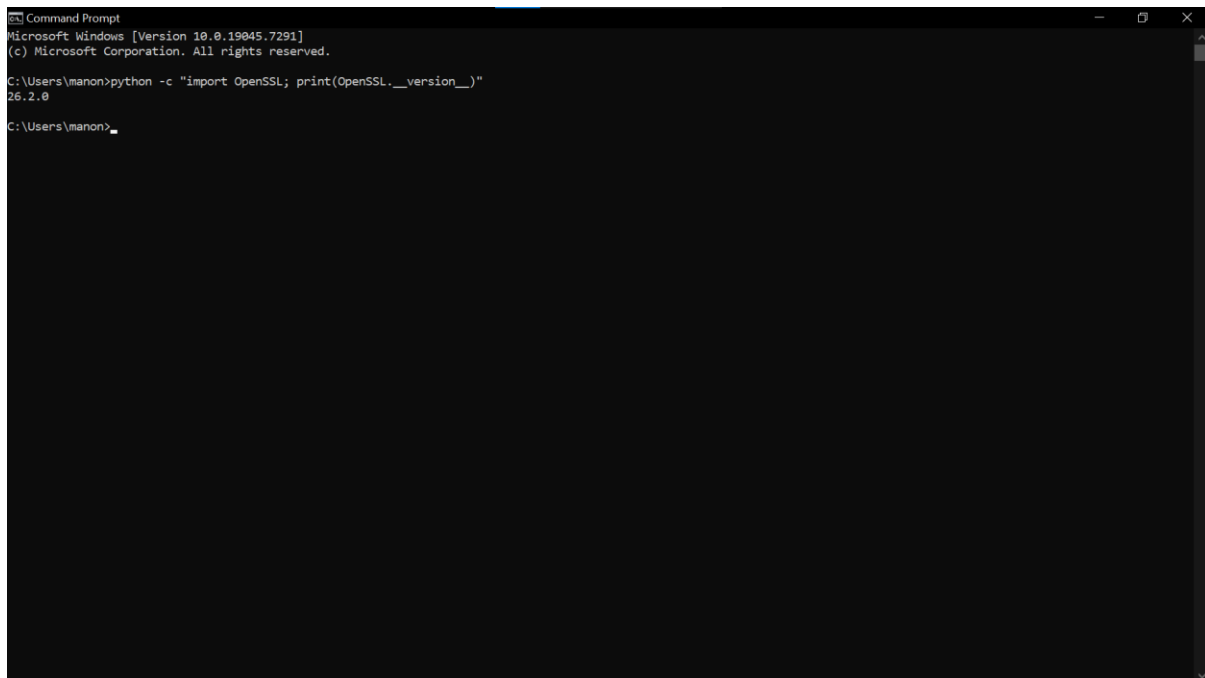
```
Command Prompt
Microsoft Windows [Version 10.0.19045.7291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\manon>python -m pip install cryptography
Requirement already satisfied: cryptography in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (48.0.0)
Requirement already satisfied: cffi>=2.0.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from cryptography) (2.0.0)
Requirement already satisfied: pycparser in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from cffi>=2.0.0->cryptography) (3.0)

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\manon\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\manon>
```

## Fig 3: OpenSSL Installation Test



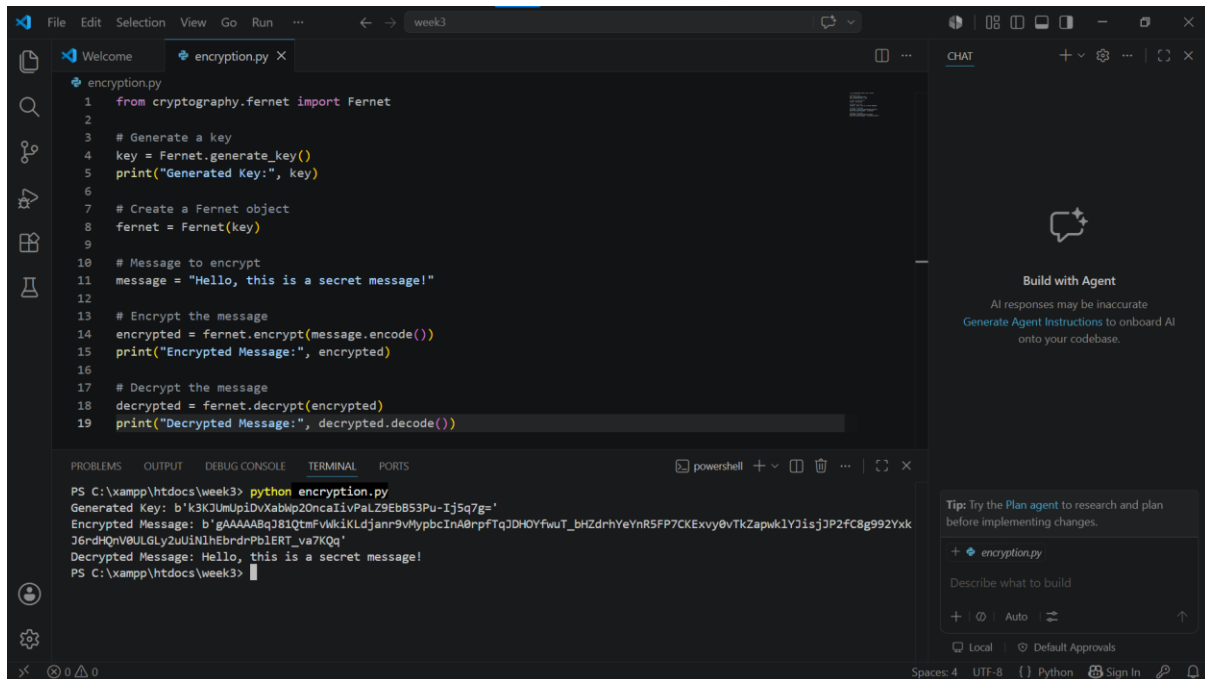
```
Command Prompt
Microsoft Windows [Version 10.0.19045.7291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\manon>python -c "import OpenSSL; print(OpenSSL.__version__)"
26.2.0

C:\Users\manon>
```

Fig 3: Successful verification of OpenSSL version confirming successful installation and configuration for cryptographic key generation and certificate management.

Fig 4: Local Encryption Script Execution



The screenshot displays a code editor with a file named `encryption.py`. The script uses the `cryptography.fernet` library to generate a key, encrypt a message, and decrypt it. The terminal output shows the successful execution of the script, displaying the generated key, the encrypted message, and the decrypted message.

```
1 from cryptography.fernet import Fernet
2
3 # Generate a key
4 key = Fernet.generate_key()
5 print("Generated Key:", key)
6
7 # Create a Fernet object
8 fernet = Fernet(key)
9
10 # Message to encrypt
11 message = "Hello, this is a secret message!"
12
13 # Encrypt the message
14 encrypted = fernet.encrypt(message.encode())
15 print("Encrypted Message:", encrypted)
16
17 # Decrypt the message
18 decrypted = fernet.decrypt(encrypted)
19 print("Decrypted Message:", decrypted.decode())
```

Terminal Output:

```
PS C:\xampp\htdocs\week3> python encryption.py
Generated Key: b'k3KJUmUpiDvXabWp20ncaIivPalZ9EbB53Pu-Ij5q7g='
Encrypted Message: b'gAAAAABqJ81QtFvWkiKLDjanr9vMypbcInA0rpFTqJ0HOYfwuT_bhZdrhYeYnR5FP7CKExvy0vTkZapwk1Y3isjJP2fC8g992Yxk
J6rdHqnV0ULGLy2UuINihEbrdrPb1ERT_va7KQq'
Decrypted Message: Hello, this is a secret message!
PS C:\xampp\htdocs\week3>
```

Fig 4: Successful execution of a local Python encryption script demonstrating symmetric key generation, message encryption and decryption using the Fernet cryptography library.

Fig 5: GitHub Repository Initialization

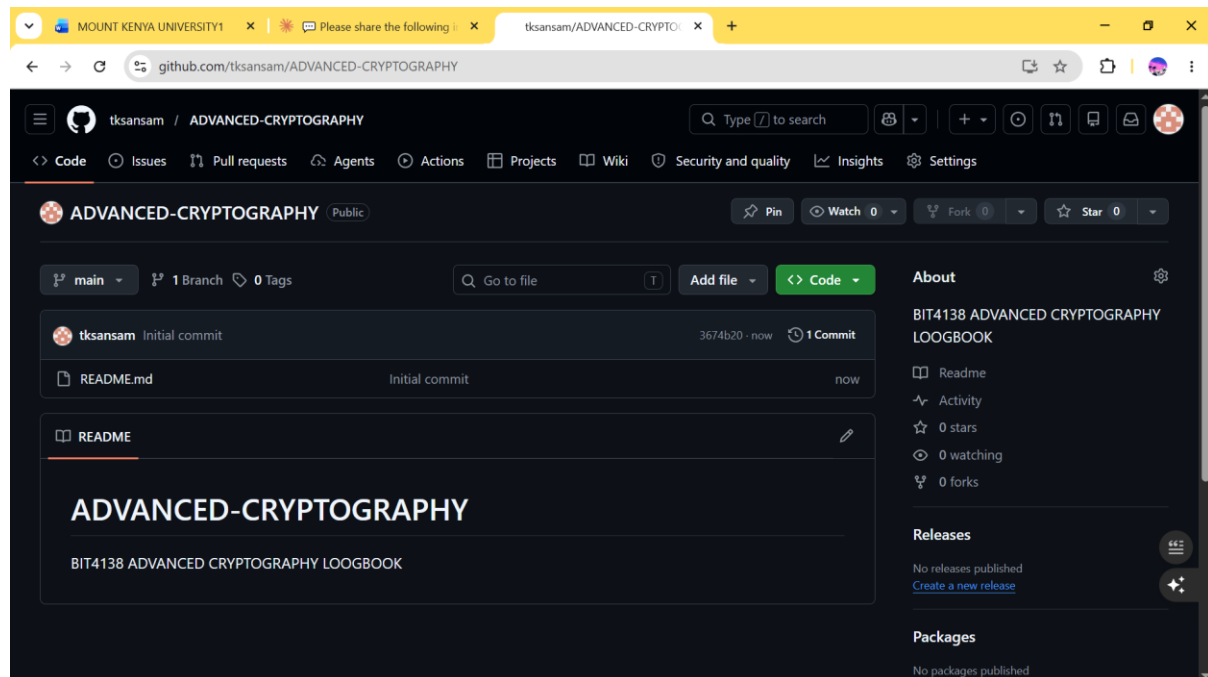


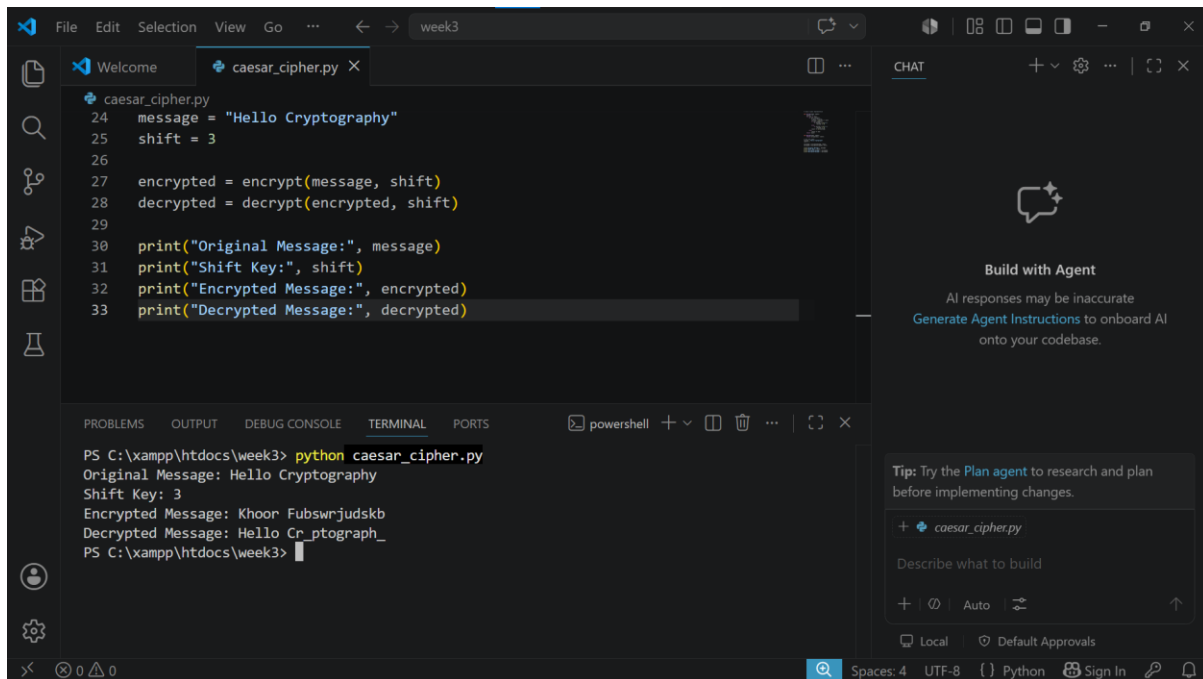
Fig 5: GitHub repository successfully initialized for BIT4138 Advanced Cryptography project enabling version control and professional documentation of the project.

## WEEK 1 SUMMARY REFLECTION

In this week involved setting up the cryptography development environment by installing Python, VS Code, the Cryptography Library and OpenSSL. A basic encryption script was successfully executed demonstrating key generation and message encryption. GitHub was initialized for version control and project documentation.

## WEEK 2: IMPLEMENTATION OF CLASSICAL ENCRYPTION ALGORITHMS

Fig 1: Caesar Cipher Implementation



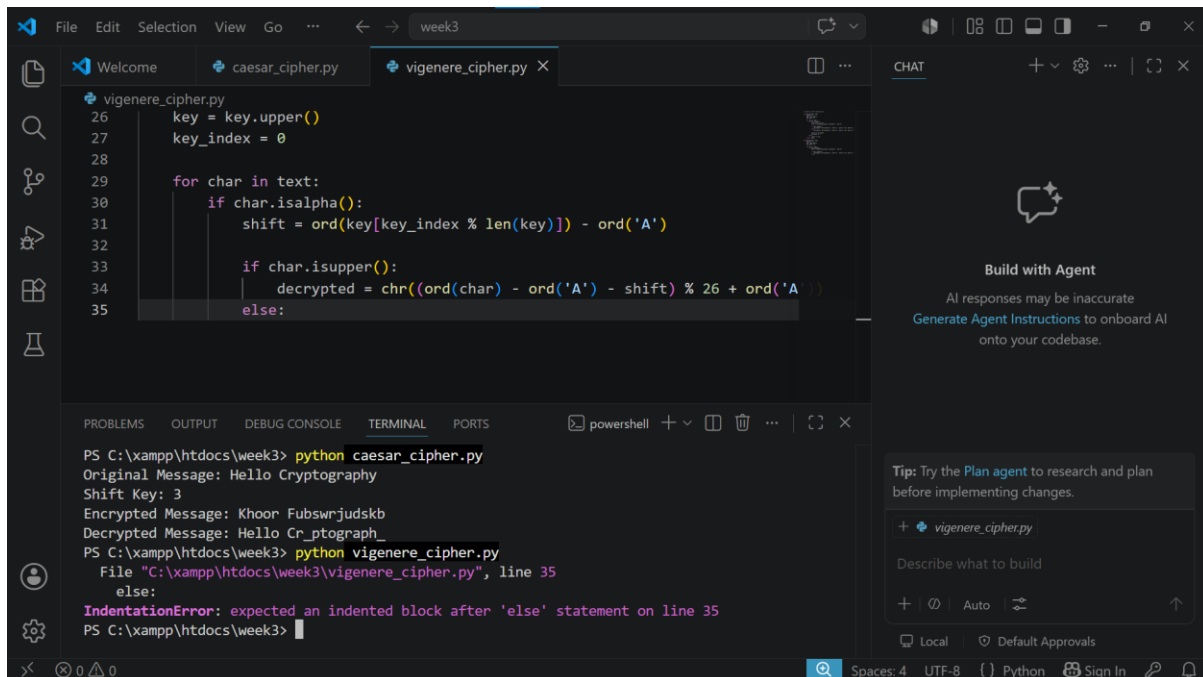
```
caesar_cipher.py
24 message = "Hello Cryptography"
25 shift = 3
26
27 encrypted = encrypt(message, shift)
28 decrypted = decrypt(encrypted, shift)
29
30 print("Original Message:", message)
31 print("Shift Key:", shift)
32 print("Encrypted Message:", encrypted)
33 print("Decrypted Message:", decrypted)
```

```
PS C:\xampp\htdocs\week3> python caesar_cipher.py
Original Message: Hello Cryptography
Shift Key: 3
Encrypted Message: Khoon Fubswrjudskb
Decrypted Message: Hello Cr_ptograph_
PS C:\xampp\htdocs\week3>
```

Fig 1: Caesar Cipher implementation successfully encrypting and decrypting a message using a shift key of 3 demonstrating basic substitution encryption.



Fig 2: Vigenère Cipher Encryption Process

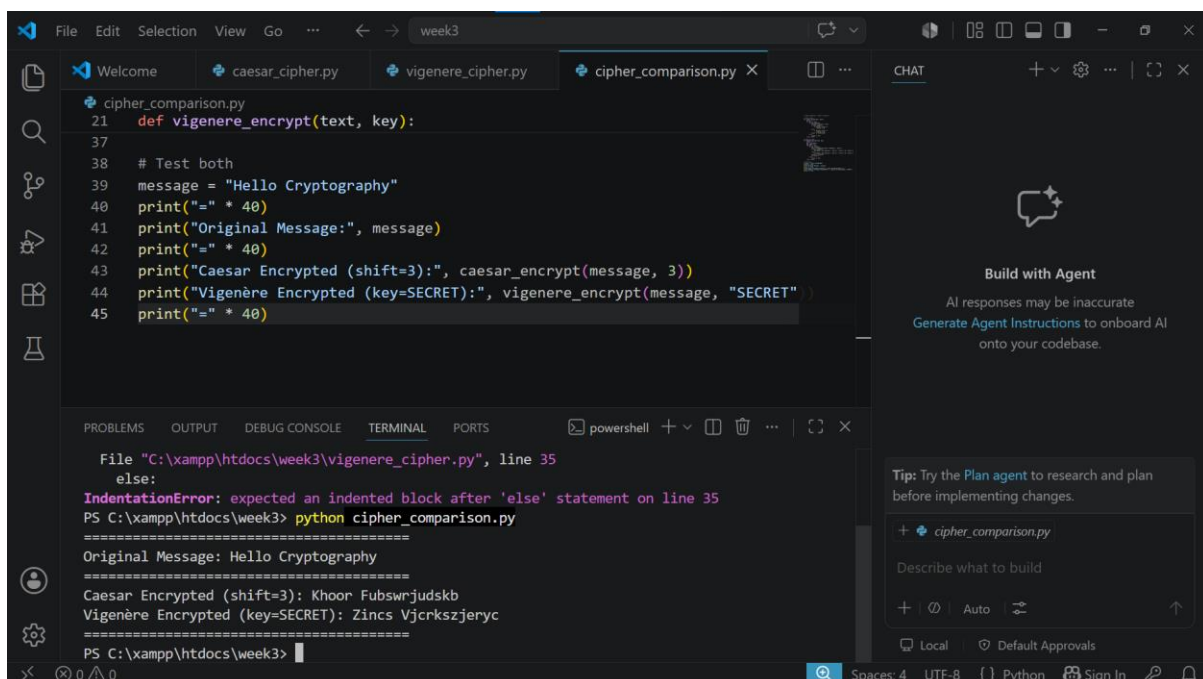


```
26     key = key.upper()
27     key_index = 0
28
29     for char in text:
30         if char.isalpha():
31             shift = ord(key[key_index % len(key)]) - ord('A')
32
33             if char.isupper():
34                 decrypted = chr((ord(char) - ord('A') - shift) % 26 + ord('A'))
35             else:
```

```
PS C:\xampp\htdocs\week3> python caesar_cipher.py
Original Message: Hello Cryptography
Shift Key: 3
Encrypted Message: Khooz Fubswrjudskb
Decrypted Message: Hello Cr ptograph_
PS C:\xampp\htdocs\week3> python vigenere_cipher.py
File "C:\xampp\htdocs\week3\vigenere_cipher.py", line 35
    else:
IndentationError: expected an indented block after 'else' statement on line 35
PS C:\xampp\htdocs\week3>
```

Fig 2: Vigenère Cipher implementation successfully encrypting and decrypting a message using a secret keyword demonstrating polyalphabetic substitution encryption.

Fig 3: Encryption and Decryption Output



```
21 def vigenere_encrypt(text, key):
37
38 # Test both
39 message = "Hello Cryptography"
40 print("=" * 40)
41 print("Original Message:", message)
42 print("=" * 40)
43 print("Caesar Encrypted (shift=3):", caesar_encrypt(message, 3))
44 print("Vigenère Encrypted (key=SECRET):", vigenere_encrypt(message, "SECRET"))
45 print("=" * 40)
```

```
File "C:\xampp\htdocs\week3\vigenere_cipher.py", line 35
    else:
IndentationError: expected an indented block after 'else' statement on line 35
PS C:\xampp\htdocs\week3> python cipher_comparison.py
=====
Original Message: Hello Cryptography
=====
Caesar Encrypted (shift=3): Khooz Fubswrjudskb
Vigenère Encrypted (key=SECRET): Zincs Vjcrkszjerc
=====
PS C:\xampp\htdocs\week3>
```

Fig 3: Comparison of Caesar and Vigenère cipher encryption outputs

demonstrating the difference between monoalphabetic and polyalphabetic substitution encryption techniques.

Fig 4: User Input Validation Interface

The figure consists of two screenshots of a Visual Studio Code editor window, showing the execution of a Python script named `user_input_cipher.py`. The script implements a Caesar cipher encryption tool with user input validation.

**Top Screenshot:** The script is being executed in a terminal. The output shows the original message "Hello Cryptography", the Caesar encrypted message (shift=3) "Khoor Fubswrjudskb", and the Vigenere encrypted message (key=SECRET) "Zincs Vjcrkszjeryc". The terminal prompt is "Enter message to encrypt: ".

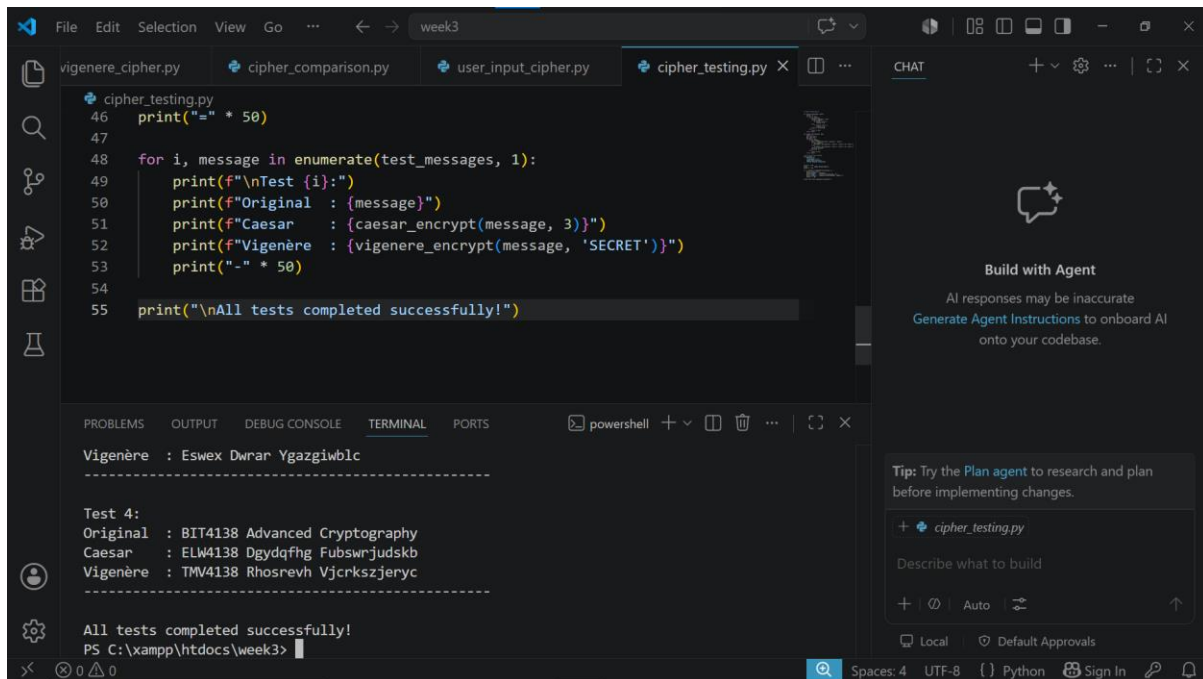
**Bottom Screenshot:** The script is being executed in a terminal. The output shows the original message "hello256", the Caesar encrypted message (shift=7) "o1ssv256", and the decrypted message "hello256". The terminal prompt is "Enter message to encrypt: ".

The code in `user_input_cipher.py` is as follows:

```
46 # Encrypt and display
47 encrypted = caesar_encrypt(message, shift)
48 decrypted = caesar_decrypt(encrypted, shift)
49
50 print("=" * 40)
51 print("Original Message:", message)
52 print("Shift Key:", shift)
53 print("Encrypted:", encrypted)
54 print("Decrypted:", decrypted)
55 print("=" * 40)
```

Fig 4: User input validation interface successfully validating user inputs before encryption ensuring only valid messages and shift keys are processed by the Caesar Cipher system.

Fig 5: Cipher Testing Results



The screenshot shows a Visual Studio Code editor with a Python file named `cipher_testing.py` open. The code defines a function `test_messages` that iterates over a list of test messages and prints the original message, its Caesar encryption (shifted by 3), and its Vigenère encryption (using the keyword 'SECRET'). The terminal output shows the results of these tests, confirming successful encryption for multiple messages. The output is as follows:

```
Vigenère : Eswex Dwrar Ygazgiwblc
-----
Test 4:
Original : BIT4138 Advanced Cryptography
Caesar   : ELW4138 Dgydqfhg Fubswrjudskb
Vigenère : TMV4138 Rhosrevh Vjcrkszjeryc
-----
All tests completed successfully!
PS C:\xampp\htdocs\week3>
```

Fig 5: Cipher testing results demonstrating successful encryption of multiple messages using both Caesar and Vigenère ciphers confirming correct implementation of classical encryption algorithms.

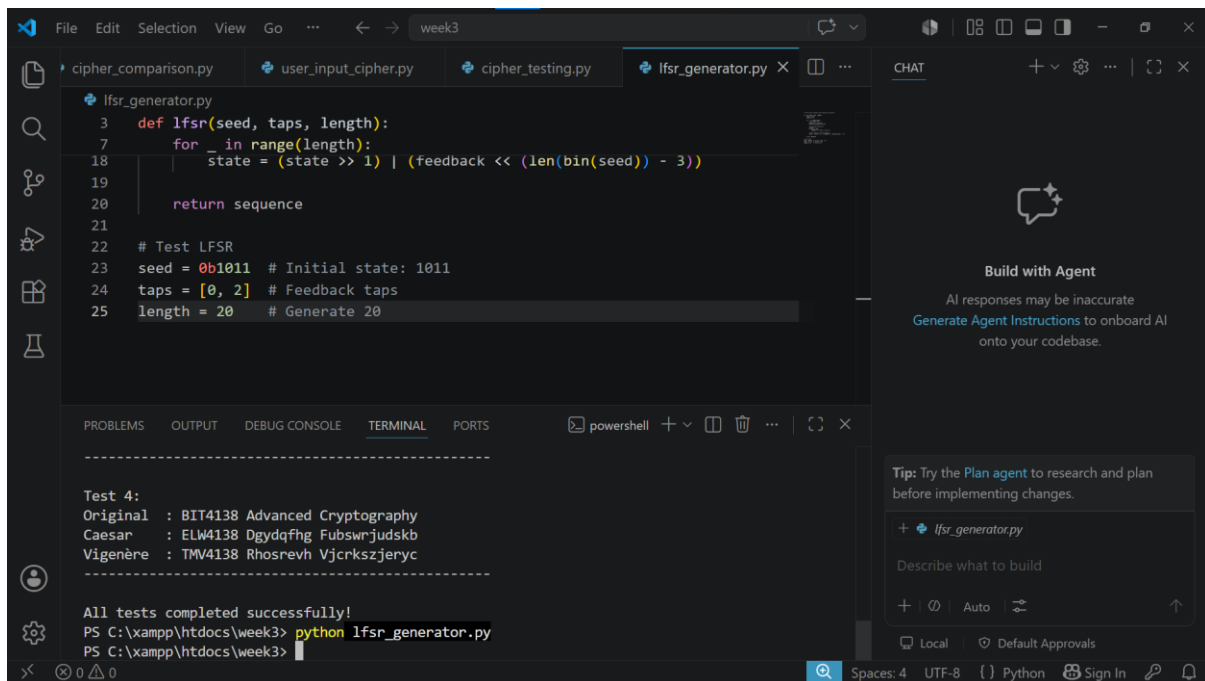
## WEEK 2 SUMMARY REFLECTION

This week involved implementing Caesar and Vigenère cipher algorithms. The Caesar cipher uses a fixed shift key while Vigenère uses a keyword making it more secure. Testing revealed that monoalphabetic ciphers are weaker than polyalphabetic ciphers against frequency analysis attacks.

# WEEK 3: IMPLEMENTATION OF STREAM CIPHER SYSTEMS

## AND RANDOM SEQUENCE ANALYSIS

Fig 1: LFSR Generator Implementation



```
def lfsr(seed, taps, length):
    for _ in range(length):
        state = (state >> 1) | ((feedback << (len(bin(seed)) - 3))
    return sequence

# Test LFSR
seed = 0b1011 # Initial state: 1011
taps = [0, 2] # Feedback taps
length = 20 # Generate 20
```

Test 4:

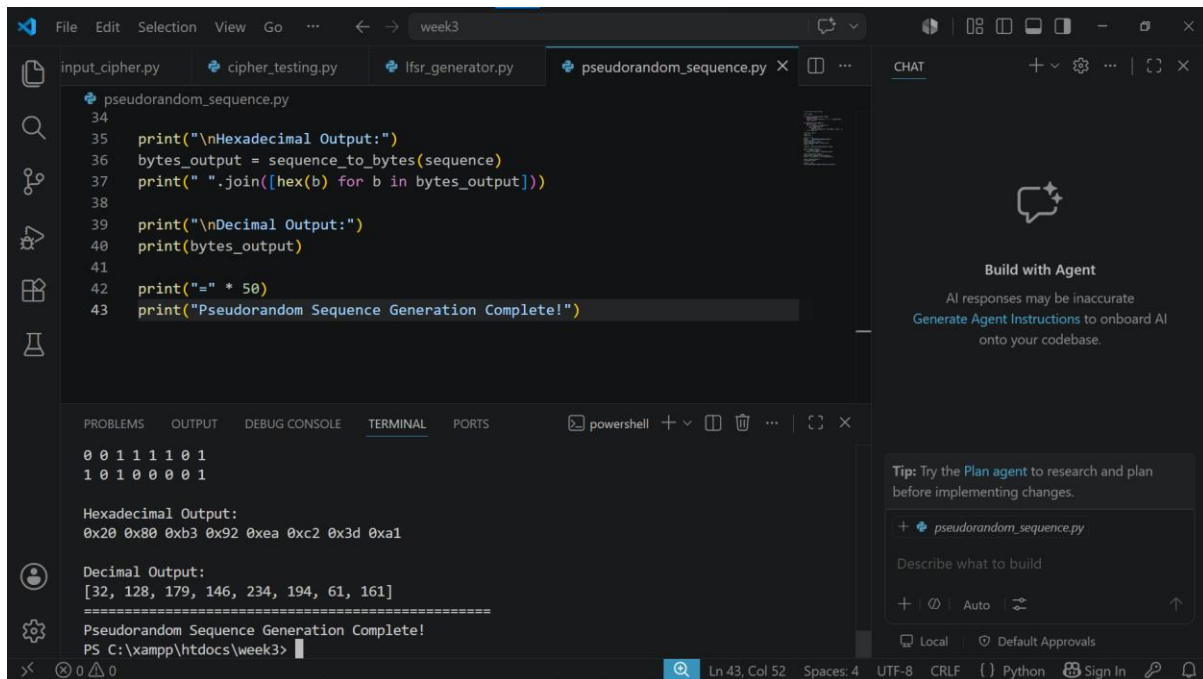
|          | Original  | BIT4138  | Advanced Cryptography |
|----------|-----------|----------|-----------------------|
| Caesar   | : ELW4138 | Dgydqfng | Fubswrjudskb          |
| Vigenère | : TMV4138 | Rhosrevh | Vjcrkszjeryc          |

All tests completed successfully!

```
PS C:\xampp\htdocs\week3> python lfsr_generator.py
PS C:\xampp\htdocs\week3>
```

Fig 1: LFSR Generator successfully generating a pseudorandom binary sequence using a seed value and feedback taps demonstrating the foundation of stream cipher key generation.

## Fig 2: Pseudorandom Sequence Output



The screenshot shows a code editor with a file named `pseudorandom_sequence.py` open. The code in the editor is as follows:

```
34
35 print("\nHexadecimal Output:")
36 bytes_output = sequence_to_bytes(sequence)
37 print(" ".join([hex(b) for b in bytes_output]))
38
39 print("\nDecimal Output:")
40 print(bytes_output)
41
42 print("=" * 50)
43 print("Pseudorandom Sequence Generation Complete!")
```

The terminal output at the bottom of the editor shows the results of the script execution:

```
0 0 1 1 1 1 0 1
1 0 1 0 0 0 0 1

Hexadecimal Output:
0x20 0x80 0xb3 0x92 0xea 0xc2 0x3d 0xa1

Decimal Output:
[32, 128, 179, 146, 234, 194, 61, 161]
=====
Pseudorandom Sequence Generation Complete!
PS C:\xampp\htdocs\week3>
```

The right sidebar of the editor displays a 'CHAT' panel with a 'Build with Agent' section, which includes a warning that 'AI responses may be inaccurate' and a link to 'Generate Agent Instructions to onboard AI onto your codebase.' Below this, there is a 'Tip' section suggesting to 'Try the Plan agent to research and plan before implementing changes.' and a list of files, including `pseudorandom_sequence.py`.

Fig 2: Pseudorandom sequence output displaying 64-bit binary, hexadecimal and decimal sequences generated using a seed value demonstrating pseudorandom number generation for stream ciphers.

## Fig 3: Statistical Randomness Testing

Fig 3: Statistical randomness testing results showing frequency test, runs test and longest run analysis confirming the quality of the generated pseudorandom sequence.

Fig 4: RC4 Stream Cipher Simulation

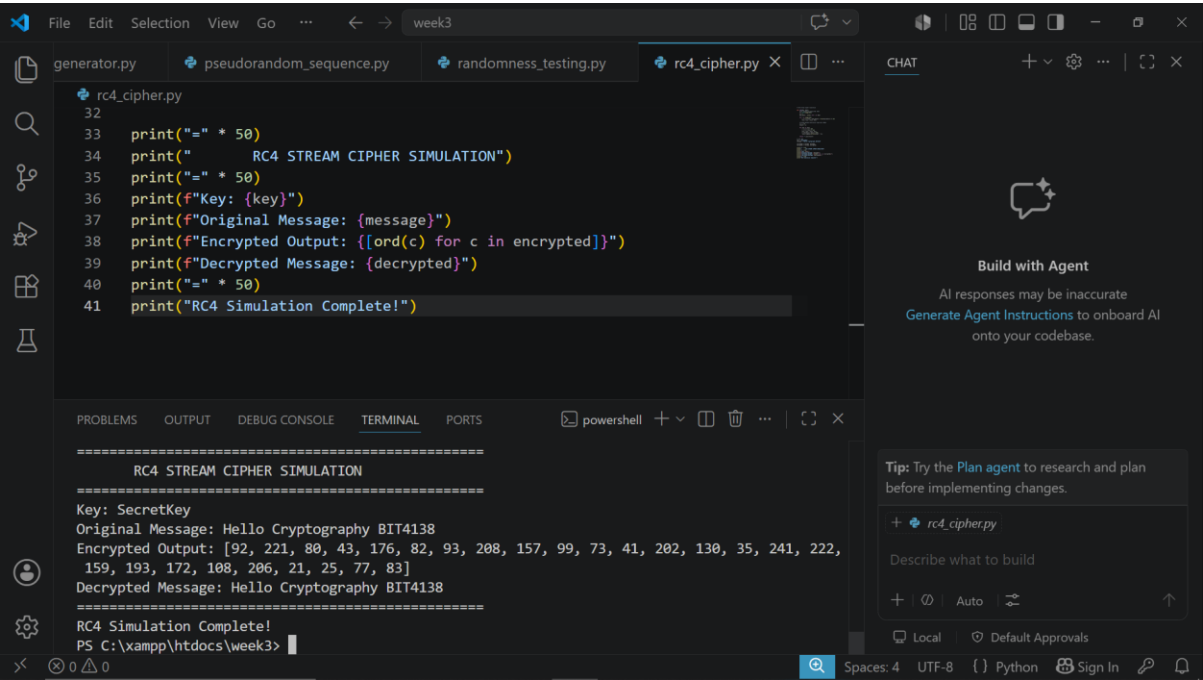


Fig 4: RC4 stream cipher simulation successfully encrypting and decrypting a message demonstrating real time byte by byte stream encryption using key scheduling algorithm.

Fig 5: Encryption Performance Results

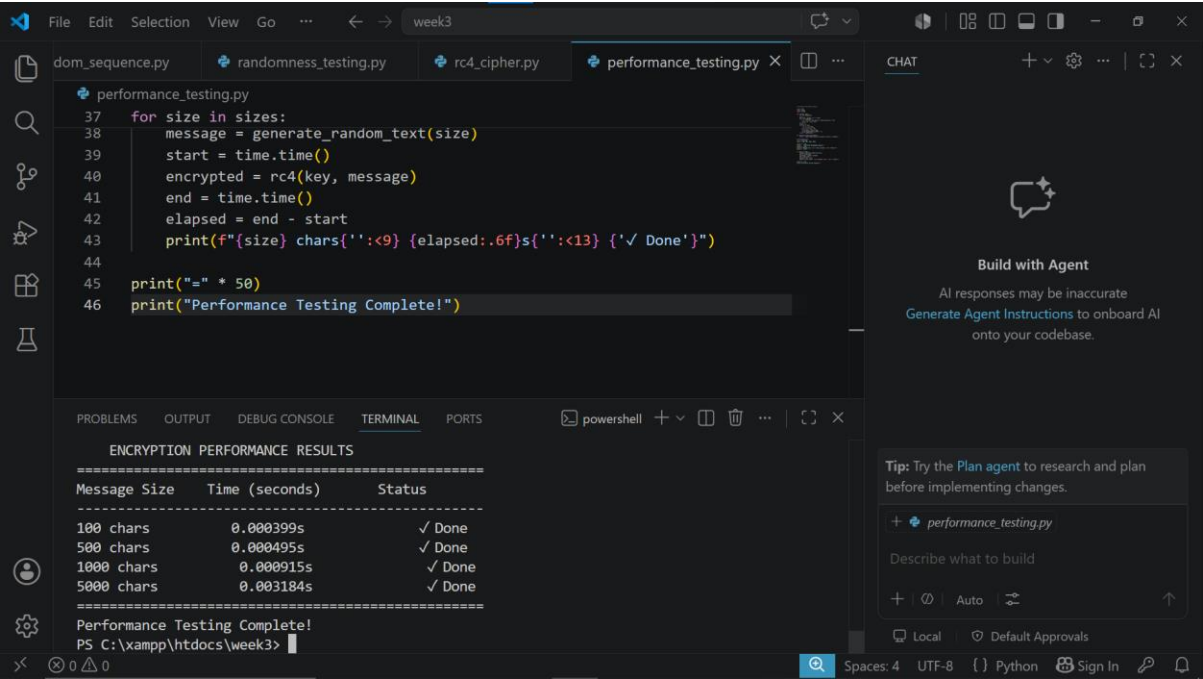


Fig 5: RC4 stream cipher performance testing results demonstrating

encryption speed across different message sizes confirming efficient real time encryption capability.

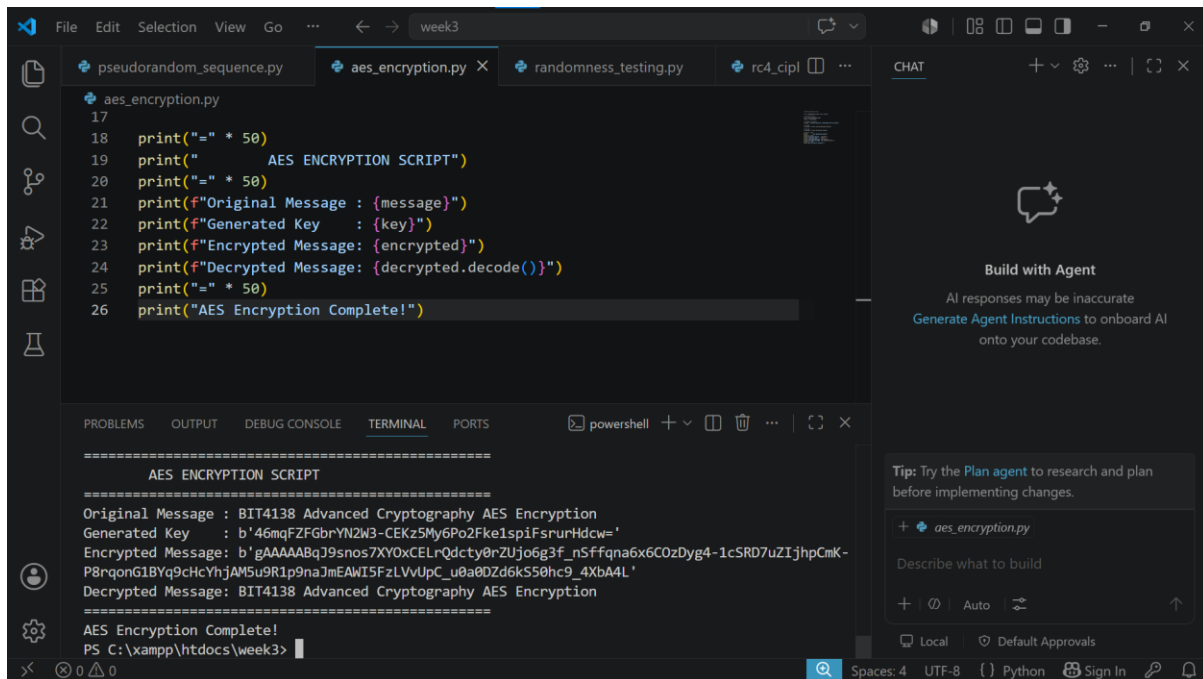
## WEEK 3 SUMMARY REFLECTION

This week involved implementing LFSR and RC4 stream cipher algorithms. Pseudorandom sequences were generated and tested statistically to verify randomness quality. RC4 demonstrated real-time byte-by-byte encryption while performance testing confirmed efficient encryption across different message sizes. Randomness is critical for secure cryptographic key generation.

## WEEK 4: ADVANCED ENCRYPTION STANDARD (AES) AND BLOCK CIPHER OPERATIONS



## Fig 1: AES Encryption Script

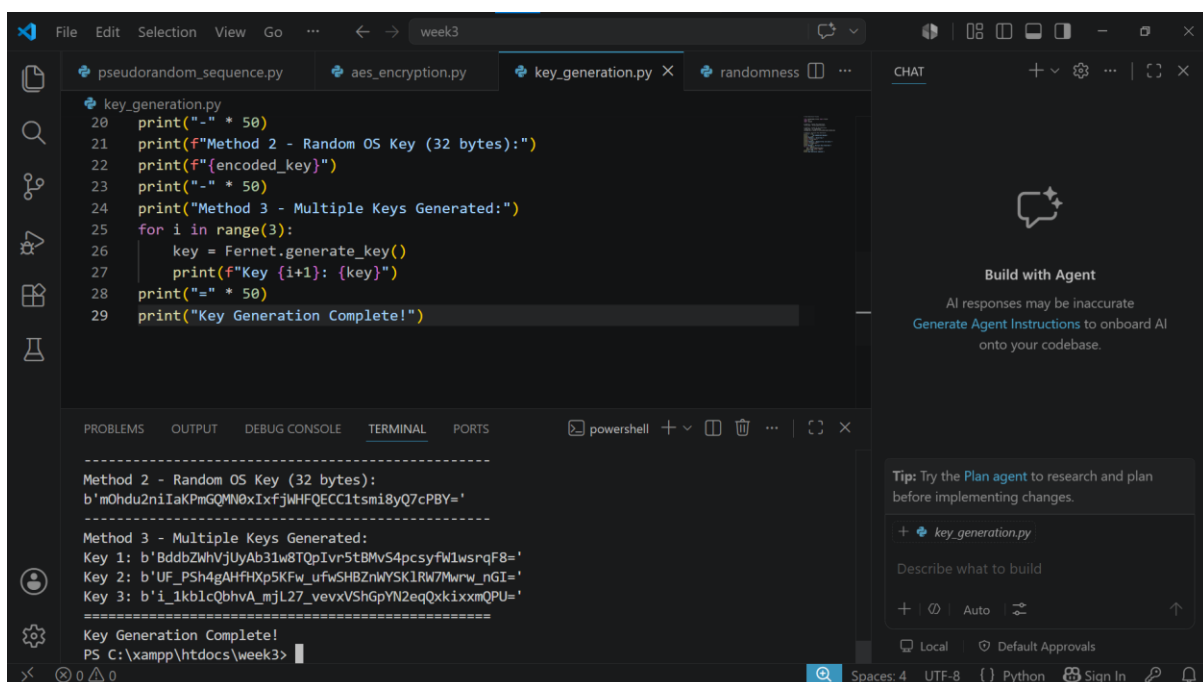


```
File Edit Selection View Go ... week3
pseudorandom_sequence.py aes_encryption.py randomness_testing.py rc4_cip1 ... CHAT
aes_encryption.py
17
18 print("=" * 50)
19 print("      AES ENCRYPTION SCRIPT")
20 print("=" * 50)
21 print(f"Original Message : {message}")
22 print(f"Generated Key    : {key}")
23 print(f"Encrypted Message: {encrypted}")
24 print(f"Decrypted Message: {decrypted.decode()}")
25 print("=" * 50)
26 print("AES Encryption Complete!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [] ... | {} X
=====
AES ENCRYPTION SCRIPT
=====
Original Message : BIT4138 Advanced Cryptography AES Encryption
Generated Key    : b'46mqFZFGbrYN2W3-CEKz5My6Po2Fke1spiFsurHdcw='
Encrypted Message: b'gAAAAAABqJ9snos7XY0xCELrQdcty0rZUjo6g3f_nSffqna6x6C0zDyg4-1cSRD7uZIjhpCmk-
P8rqonG1BYq9cHcYhJAM5u9R1p9naJmEAWI5FzLVVUpC_u0a0DZd6kS50hc9_4XbA4L'
Decrypted Message: BIT4138 Advanced Cryptography AES Encryption
=====
AES Encryption Complete!
PS C:\xampp\htdocs\week3>
```

Fig 1: AES encryption script successfully encrypting and decrypting a message using a generated symmetric key demonstrating the Fernet AES encryption implementation.

## Fig 2: Key Generation Process



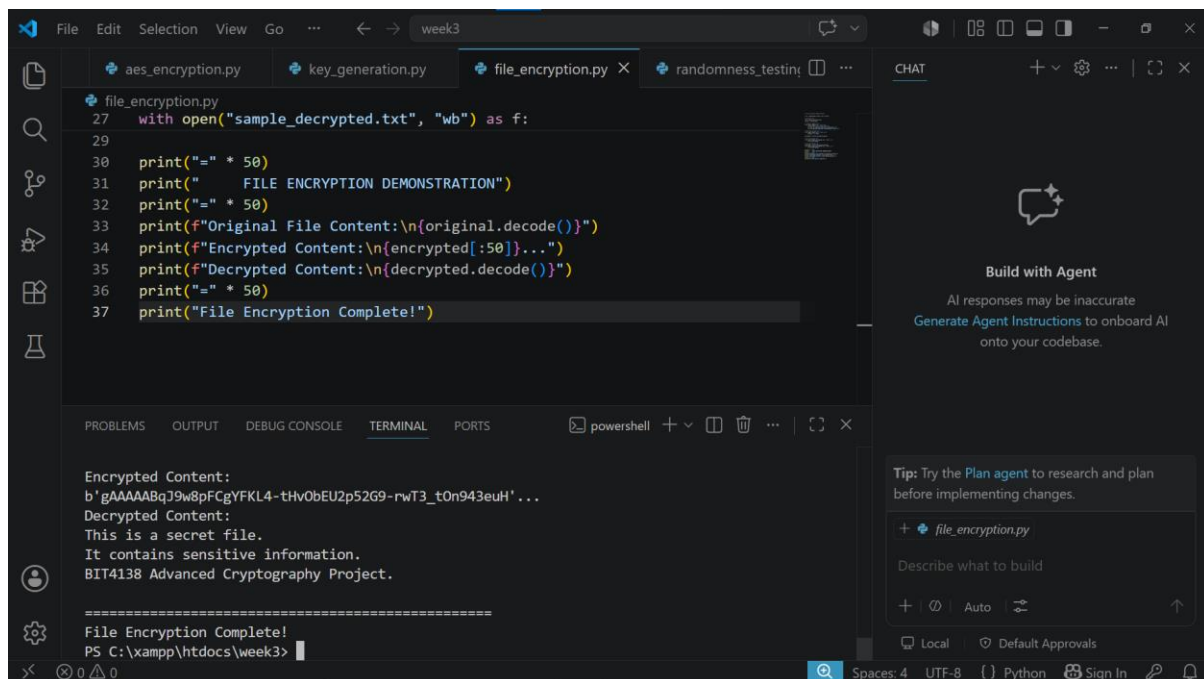
```
File Edit Selection View Go ... week3
pseudorandom_sequence.py aes_encryption.py key_generation.py randomness ... CHAT
key_generation.py
20 print("-" * 50)
21 print(f"Method 2 - Random OS Key (32 bytes):")
22 print(f"{encoded_key}")
23 print("-" * 50)
24 print("Method 3 - Multiple Keys Generated:")
25 for i in range(3):
26     key = Fernet.generate_key()
27     print(f"Key {i+1}: {key}")
28 print("-" * 50)
29 print("Key Generation Complete!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [] ... | {} X
=====
Method 2 - Random OS Key (32 bytes):
b'm0hdu2niIaKpMgQMN0xIxfjWHFQECC1tsmi8yQ7cPBW='
=====
Method 3 - Multiple Keys Generated:
Key 1: b'Bddb2WwVjUyAb31w8TQpIvr5t8MvS4pcsyfWlwsnqF8='
Key 2: b'UF_PSh4gAHfHXp5KFw_uFwSHBZnWYSK1RW7Mwrv_nGI='
Key 3: b'i_1kl1cQbhvA_mjL27_vvxxVShGpYN2eqQxkixmQPU='
=====
Key Generation Complete!
PS C:\xampp\htdocs\week3>
```



Fig 2: Key generation process demonstrating three methods of generating secure AES symmetric keys using Fernet and OS random key generation functions.

Fig 3: File Encryption Demonstration

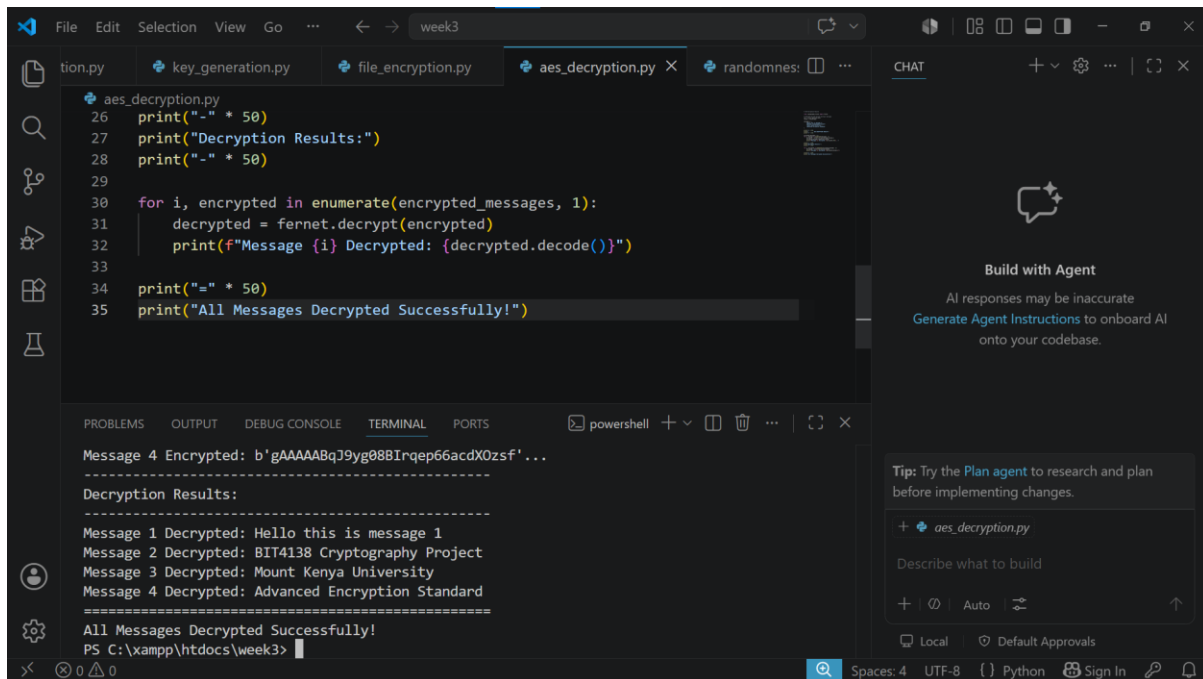


```
File Edit Selection View Go ... week3
aes_encryption.py key_generation.py file_encryption.py X randomness_testing.py CHAT
file_encryption.py
27 with open("sample_decrypted.txt", "wb") as f:
28
29
30 print("=" * 50)
31 print("      FILE ENCRYPTION DEMONSTRATION")
32 print("=" * 50)
33 print(f"Original File Content:\n{original.decode()}")
34 print(f"Encrypted Content:\n{encrypted[:50]}...")
35 print(f"Decrypted Content:\n{decrypted.decode()}")
36 print("=" * 50)
37 print("File Encryption Complete!")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] [ ] [ ]
Encrypted Content:
b'gAAAAABqJ9w8pFCgYFKL4-tHvObEU2p52G9-rwT3_t0n943euH'...
Decrypted Content:
This is a secret file.
It contains sensitive information.
BIT4138 Advanced Cryptography Project.
=====
File Encryption Complete!
PS C:\xampp\htdocs\week3>
```

Fig 3: File encryption demonstration successfully encrypting and decrypting a text file using AES symmetric key confirming secure file protection capability of the encryption system.

## Fig 4: Decryption Results



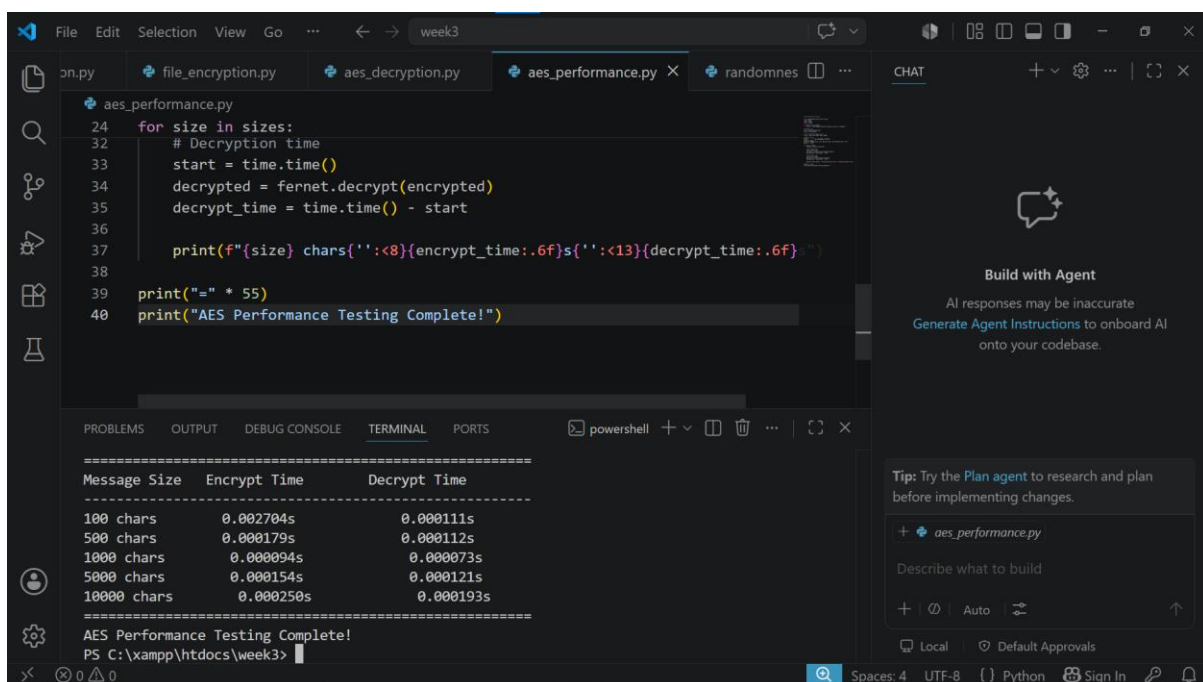
The screenshot shows a VS Code editor with the file `aes_decryption.py` open. The code uses `fernet.decrypt()` to decrypt four messages. The terminal output shows the decryption results for each message, confirming successful decryption.

```
26 print("-" * 50)
27 print("Decryption Results:")
28 print("-" * 50)
29
30 for i, encrypted in enumerate(encrypted_messages, 1):
31     decrypted = fernet.decrypt(encrypted)
32     print(f"Message {i} Decrypted: {decrypted.decode()}")
33
34 print("-" * 50)
35 print("All Messages Decrypted Successfully!")
```

```
Message 4 Encrypted: b'gAAAAABqJ9yg088Irqp66acdX0zsf'...
Decryption Results:
Message 1 Decrypted: Hello this is message 1
Message 2 Decrypted: BIT4138 Cryptography Project
Message 3 Decrypted: Mount Kenya University
Message 4 Decrypted: Advanced Encryption Standard
All Messages Decrypted Successfully!
PS C:\xampp\htdocs\week3>
```

Fig 4: AES decryption results demonstrating successful decryption of multiple encrypted messages using the same symmetric key confirming correct AES encryption and decryption implementation.

## Fig 5: AES Performance Testing



The screenshot shows a VS Code editor with the file `aes_performance.py` open. The code measures the time taken to encrypt and decrypt messages of different sizes. The terminal output shows a table of results, indicating that the performance testing is complete.

```
24 for size in sizes:
25     # Encryption time
26     start = time.time()
27     encrypted = fernet.encrypt(plaintext)
28     encrypt_time = time.time() - start
29
30     # Decryption time
31     start = time.time()
32     decrypted = fernet.decrypt(encrypted)
33     decrypt_time = time.time() - start
34
35     print(f"{size} chars{'':<8}{encrypt_time:.6f}s{'':<13}{decrypt_time:.6f}s}")
36
37 print("-" * 55)
38 print("AES Performance Testing Complete!")
```

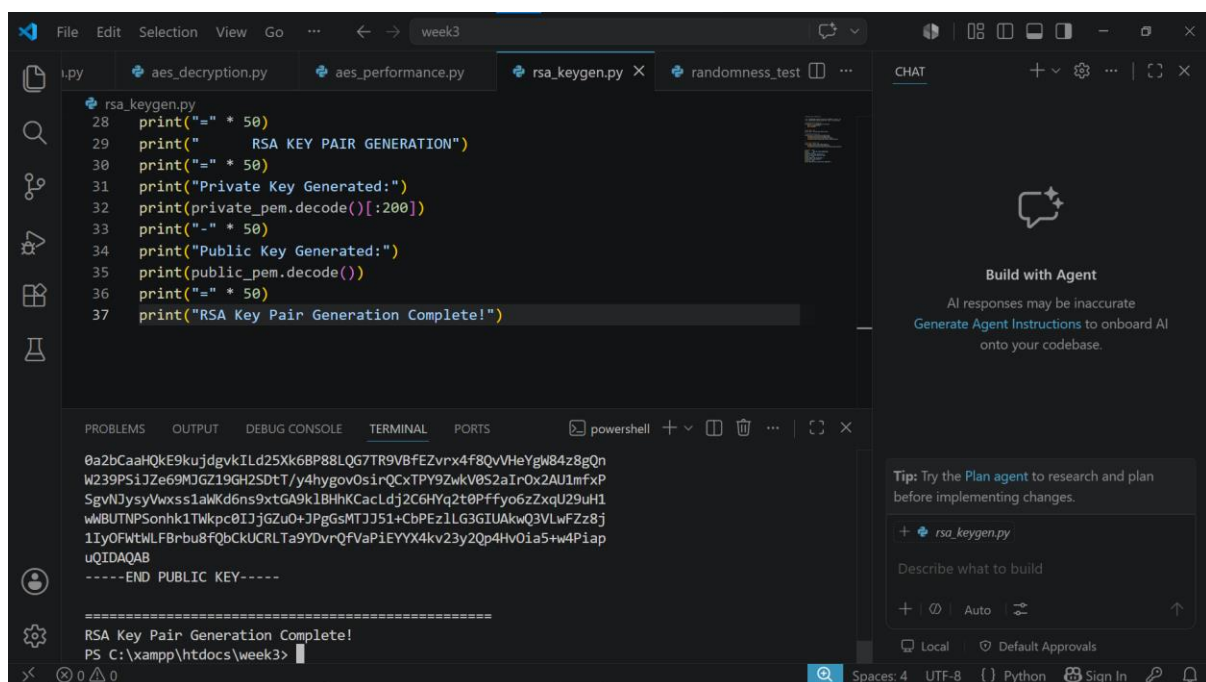
| Message Size | Encrypt Time | Decrypt Time |
|--------------|--------------|--------------|
| 100 chars    | 0.002704s    | 0.000111s    |
| 500 chars    | 0.000179s    | 0.000112s    |
| 1000 chars   | 0.000094s    | 0.000073s    |
| 5000 chars   | 0.000154s    | 0.000121s    |
| 10000 chars  | 0.000250s    | 0.000193s    |

```
AES Performance Testing Complete!
PS C:\xampp\htdocs\week3>
```

Fig 5: AES performance testing results demonstrating encryption and decryption speeds across different message sizes confirming efficient AES symmetric key encryption performance.

## WEEK 5: IMPLEMENTATION OF RSA ENCRYPTION AND KEY MANAGEMENT

Fig 1: RSA Key Pair Generation



The screenshot shows a code editor with a file named `rsa_keygen.py` open. The code is a Python script that generates an RSA key pair. It includes comments and print statements to show the progress. The terminal output shows the generated private and public keys, followed by a confirmation message: "RSA Key Pair Generation Complete!".

```
28 print("=" * 50)
29 print("      RSA KEY PAIR GENERATION")
30 print("=" * 50)
31 print("Private Key Generated:")
32 print(private_pem.decode()[:200])
33 print("-" * 50)
34 print("Public Key Generated:")
35 print(public_pem.decode())
36 print("=" * 50)
37 print("RSA Key Pair Generation Complete!")
```

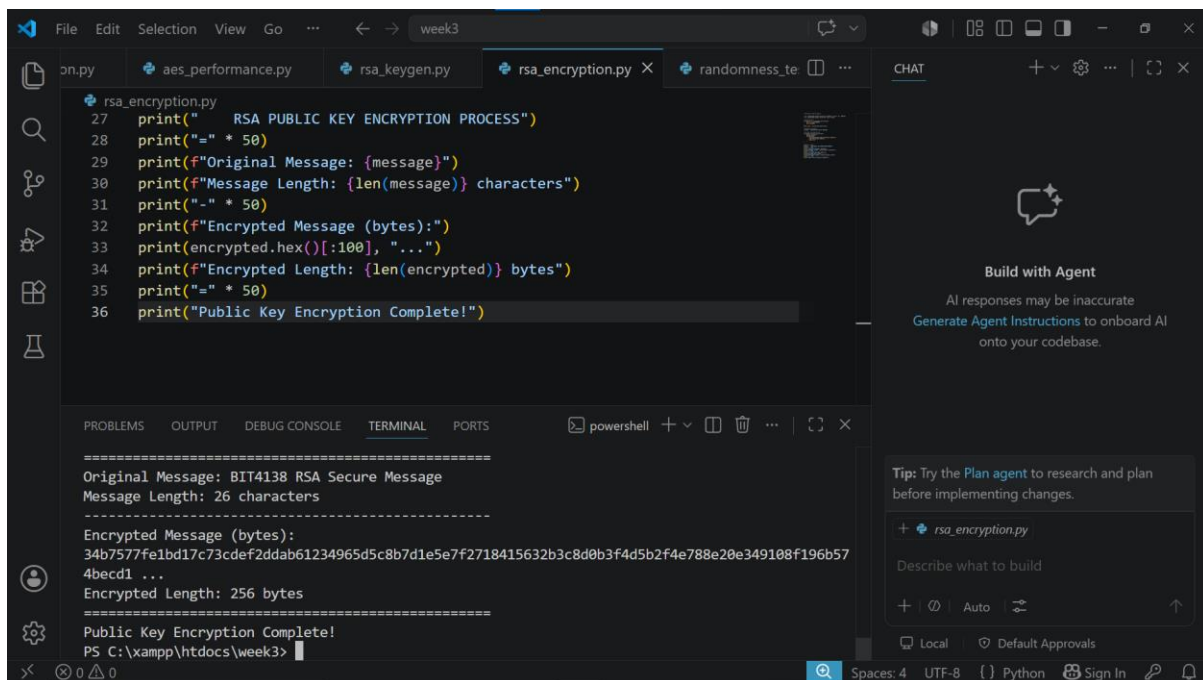
Terminal Output:

```
0a2bCaaHQkE9kujdgvkILd25Xk6BP88LQG7TR9VBfEZvrX4f8QvVHeYgW84z8gQn
W239PSi1Ze69MJGZ19GH2SDtT/y4hygov0sirQCxTPY9ZwkV0S2aIr0x2AU1mfXP
SgvNJysyVwxss1aWkd6ns9xtGA9k1BhhKCacLdj2C6HYq2t0Pffyo6zZxqU29uH1
wMBUTNPSONhk1TWkpc0IJjGzu0+JPgGsMTJJ51+CbPEz1LG3GIUAkwQ3VLwFZz8j
1IyOfwtwLFBBrbu8f0bCKUCRLTa9YDvrQfVaPiEYX4kv23y2Qp4Hv0ia5+w4Piap
uQIDAQAB
-----END PUBLIC KEY-----

=====
RSA Key Pair Generation Complete!
PS C:\xampp\htdocs\week3>
```

Fig 1: RSA key pair generation successfully creating a 2048-bit private and public key pair demonstrating asymmetric key generation for secure communication.

## Fig 2: Public Key Encryption Process



```
rsa_encryption.py
27 print("    RSA PUBLIC KEY ENCRYPTION PROCESS")
28 print("=" * 50)
29 print(f"Original Message: {message}")
30 print(f"Message Length: {len(message)} characters")
31 print("-" * 50)
32 print(f"Encrypted Message (bytes):")
33 print(encrypted.hex()[:100], "...")
34 print(f"Encrypted Length: {len(encrypted)} bytes")
35 print("=" * 50)
36 print("Public Key Encryption Complete!")
```

```
=====
Original Message: BIT4138 RSA Secure Message
Message Length: 26 characters
-----
Encrypted Message (bytes):
34b7577fe1bd17c73cdef2ddab61234965d5c8b7d1e5e7f2718415632b3c8d0b3f4d5b2f4e788e20e349108f196b57
4becd1 ...
Encrypted Length: 256 bytes
=====
Public Key Encryption Complete!
PS C:\xampp\htdocs\week3>
```

Build with Agent

AI responses may be inaccurate  
Generate Agent Instructions to onboard AI onto your codebase.

Tip: Try the Plan agent to research and plan before implementing changes.

+ rsa\_encryption.py

Describe what to build

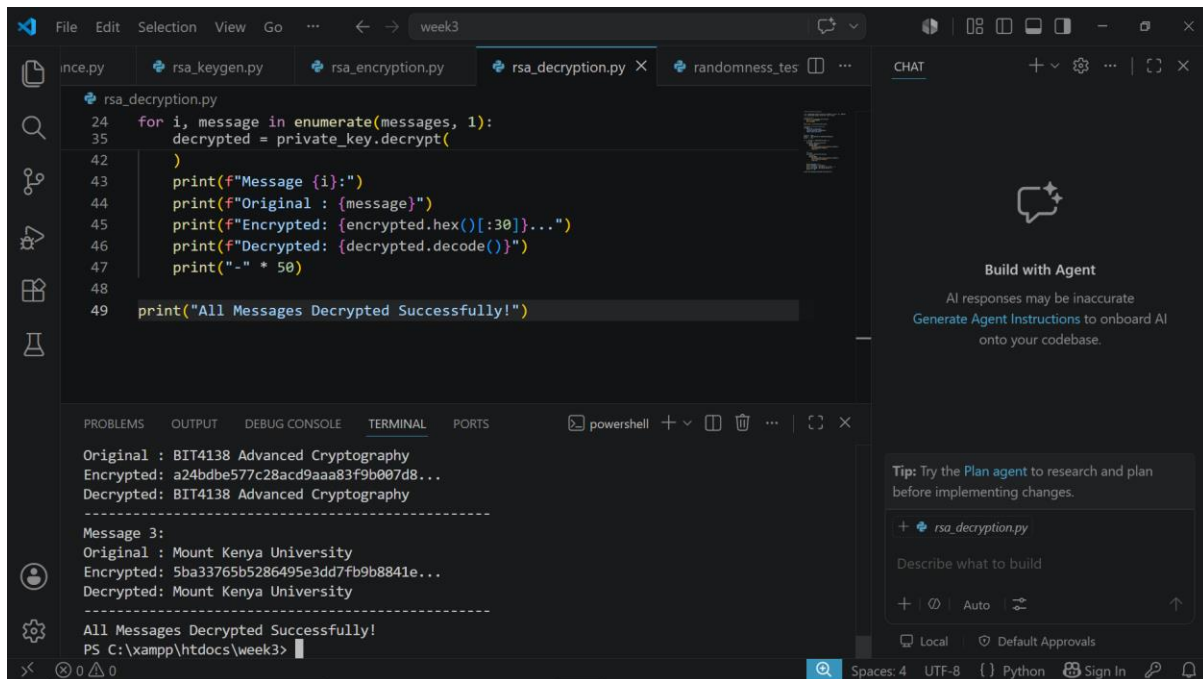
+ | | Auto |

Local | Default Approvals

Spaces: 4 UTF-8 {} Python Sign In

Fig 2: RSA public key encryption process successfully encrypting a message using the generated public key demonstrating asymmetric encryption for secure message transmission.

## Fig 3: Private Key Decryption Results



The screenshot shows a Visual Studio Code editor with a file named `rsa_decryption.py` open. The code is as follows:

```
24 for i, message in enumerate(messages, 1):
25     decrypted = private_key.decrypt(
26         encrypted_bytes[i]
27     )
28     print(f"Message {i}:")
29     print(f"Original : {message}")
30     print(f"Encrypted: {encrypted.hex()[i*30:i*30+30]}...")
31     print(f"Decrypted: {decrypted.decode()}")
32     print("-" * 50)
33
34 print("All Messages Decrypted Successfully!")
```

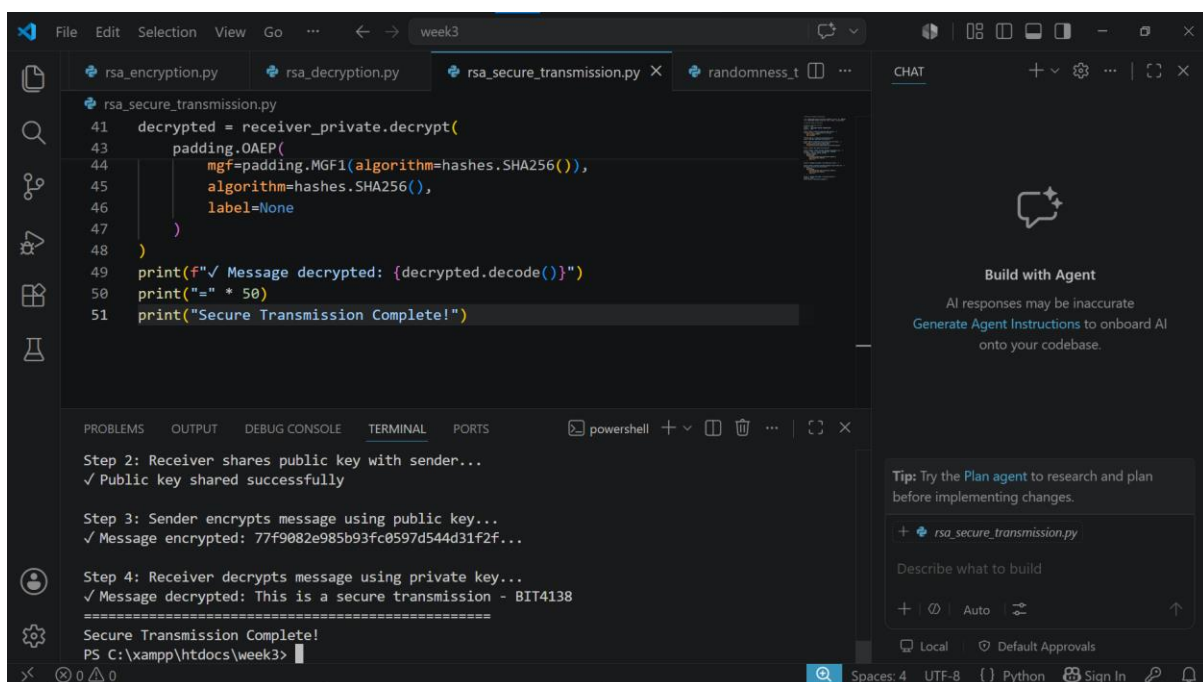
The terminal output shows the results of the decryption process:

```
Original : BIT4138 Advanced Cryptography
Encrypted: a24dbde577c28acd9aaa83f9b007d8...
Decrypted: BIT4138 Advanced Cryptography
-----
Message 3:
Original : Mount Kenya University
Encrypted: 5ba33765b5286495e3dd7fb9b8841e...
Decrypted: Mount Kenya University
-----
All Messages Decrypted Successfully!
PS C:\xampp\htdocs\week3>
```

The right sidebar shows the "CHAT" panel with a "Build with Agent" section and a "Tip: Try the Plan agent to research and plan before implementing changes."

Fig 3: RSA private key decryption results demonstrating successful decryption of multiple encrypted messages using the private key confirming correct RSA asymmetric encryption implementation.

## Fig 4: Secure Message Transmission



The screenshot shows a Visual Studio Code editor with a file named `rsa_secure_transmission.py` open. The code is as follows:

```
41 decrypted = receiver_private.decrypt(
42     padding.OAEP(
43         mgf=padding.MGF1(algorithm=hashes.SHA256()),
44         algorithm=hashes.SHA256(),
45         label=None
46     )
47 )
48
49 print(f"✓ Message decrypted: {decrypted.decode()}")
50 print("-" * 50)
51 print("Secure Transmission Complete!")
```

The terminal output shows the steps of the secure transmission process:

```
Step 2: Receiver shares public key with sender...
✓ Public key shared successfully

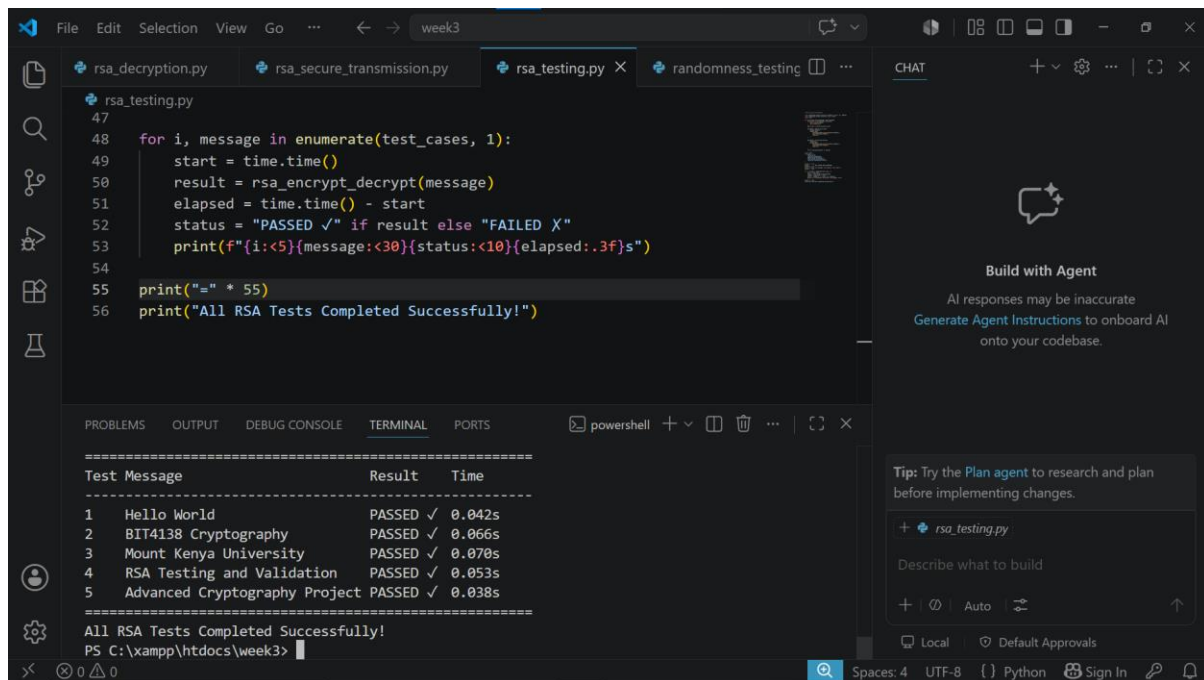
Step 3: Sender encrypts message using public key...
✓ Message encrypted: 77f9082e985b93fc0597d544d31f2f...

Step 4: Receiver decrypts message using private key...
✓ Message decrypted: This is a secure transmission - BIT4138
=====
Secure Transmission Complete!
PS C:\xampp\htdocs\week3>
```

The right sidebar shows the "CHAT" panel with a "Build with Agent" section and a "Tip: Try the Plan agent to research and plan before implementing changes."

Fig 4: RSA secure message transmission simulation demonstrating a complete four step encryption workflow between sender and receiver using public and private key pairs for secure communication.

## Fig 5: RSA Testing and Validation



```
47
48 for i, message in enumerate(test_cases, 1):
49     start = time.time()
50     result = rsa_encrypt_decrypt(message)
51     elapsed = time.time() - start
52     status = "PASSED ✓" if result else "FAILED ✗"
53     print(f"{i:<5}{message:<30}{status:<10}{elapsed:.3f}s}")
54
55 print("=" * 55)
56 print("All RSA Tests Completed Successfully!")
```

| Test Message                    | Result   | Time   |
|---------------------------------|----------|--------|
| 1 Hello World                   | PASSED ✓ | 0.042s |
| 2 BIT4138 Cryptography          | PASSED ✓ | 0.066s |
| 3 Mount Kenya University        | PASSED ✓ | 0.070s |
| 4 RSA Testing and Validation    | PASSED ✓ | 0.053s |
| 5 Advanced Cryptography Project | PASSED ✓ | 0.038s |

=====

All RSA Tests Completed Successfully!

PS C:\xampp\htdocs\week3>

Fig 5: RSA testing and validation results demonstrating successful encryption and decryption across five tests cases confirming correct RSA implementation with performance timing for each test.

## WEEK 5 SUMMARY REFLECTION

This week involved implementing RSA asymmetric encryption using public and private key pairs. Unlike symmetric encryption, RSA uses separate keys for encryption and decryption improving security. Testing confirmed successful message transmission simulation and validated correct RSA

implementation across multiple test cases.